

ASSIGNMENT INSTRUCTIONS Please read the following instructions carefully. Failure to follow the instructions will result in losing at least 5 points.

- Even though this assignment does not build on the two previous assignments, **You should continue to work in your team from assignment 2.** Here are the general rules for teams:
 1. You should work on the whole assignment together.
 2. You should **not** just partition the work between you.
 3. You should upload only one submission to canvas.
 4. You should form the pair on Canvas prior to the deadline.
- Hand in all program header/source files soft-copy using Canvas and be sure that they properly execute. Failure to do so will mean that your program is not graded.
- All header/source files should be compressed into a *ZIP* archive using the following naming format: `firstname1_lastname1_firstname2_lastname2_asmt4.zip`.
- Your ZIP file should include: (1) folder **Part1**, and (2) folder **Part3**.
- Please include your answers to all discussion questions in a single PDF file that you will submit **through Canvas**. Do **not** include it in the **ZIP** file.
This will include: Part 1 questions 2 and 3, Part 2 and Part 3 question 3.
- Please include your full name(s) in the PDF.
- **Please refer to the university integrity policy on the syllabus, and remember that you should be able to explain and reproduce any code you write and submit as part of this assignment.**

ASSIGNMENT GOALS AND OBJECTIVES

1. Programming methodology:
 - Understand the difference between various sorting algorithms.
2. C++ implementation:
 - Implement recursive functions.
 - Implement sorting algorithms.

PART 1: RECURSIVE FUNCTION – 50 points

Use the starter code provided in the folder **Part1**. Do not modify the **main** function.

Consider the following sequence, where each term is defined by its three previous terms. The second term is multiplied by two, and the third term is multiplied by 4.

$$J(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ 1 & \text{if } n = 2 \\ J(n-1) + 2 \times J(n-2) + 4 \times J(n-3) & \text{if } n > 2 \end{cases}$$

1. (25 points) Write a recursive function to compute the n -th number of the sequence. Since this sequence grows exponentially, you will quickly reach the limit of what a regular `int` variable can store. To support larger values, your return type should be a `long long` (typically a 64-bit integer allowing you to store values up to 9,223,372,036,854,775,807).
2. (10 points) Manually draw the call tree for the recursive function for $n = 6$ and include it in your PDF.
3. As discussed in class, a recursive function can be very inefficient with a large call stack. This function is also inefficient in that it computes the same value multiple times. To remove this redundancy, we use the technique called **memoization** (this is not a typo :)).
 - (a) (5 points) Define *memoization* in your own words? Please cite your sources.
 - (b) (10 points) In bullet points, briefly describe how you would implement memoization in your program.

PART 2: SORTING – 20 points + 5 EC points

Choose a non-trivial array of 8 random integers then sort it using three of the following algorithms. Show the contents of the array and program variables at each iteration of the algorithm while sorting the array into ascending order.

- (10 points) Choose **one** of the Basic Algorithms:
 1. Selection Sort
 2. Bubble Sort
 3. Insertion Sort
- (10 points) Choose **one** of these two faster algorithms:
 1. Merge Sort
 2. Quick Sort
- **Extra credit** – (5 points) one of the algorithms we have not covered in class:

1. Heap sort
2. Shell sort

You may type your answer (preferred) or write it clearly and attach all scans/photos in one PDF.

PART 3: RECURSIVE SORTING – 50 points + 10 EC points

Using the starter code provided in the folder **Part3** and the pseudo code discussed in class, **implement merge sort** in the **LinkedList** data structure, which we have been using for the **Amazon340** application.

1. (10 points) Use GenAI to generate an implementation of merge sort for a **linked list**, not for **LinkedList**. Save this code into a file **mergeSortGenAI.cpp** and include it in the submission ZIP file. See **quickSortGenAI.cpp** for an example.
2. (25 points) Using the code generated by GenAI as a starter code, write your own implementation of merge sort in the **LinkedList** data structure.
For this part:
 - you should modify/add the prototype for merge sort in **LinkedList.h**, then add the implementation to **LinkedList.cpp**.
 - You do not need to make any changes to **linkedListSortingMain.cpp**.
 - You do not have to incorporate it with the **Amazon340** application.
3. Briefly answer the following questions:
 - (a) (10 points) How did you check GenAI's code for correctness and hallucination?
 - (b) (5 points) What changes did you make to the code to adapt it to the **LinkedList** data structure?

Extra Credit (10 points)

For extra credit, implement **quick sort** in **LinkedList**. **quickSortGenAI.cpp** is provided for this part.